Video script: Brute-Force TSP, Cagayan de Oro tourist day tour

▶ Slide: change slide ▶ Action: do this on screen Normal text: say this

Target length: about 13 minutes (the brief allows 10–15). How to read this: plain text is what you **say**. Lines in `[brackets]` are what you **do**. Times are approximate. You don't have to memorize it; read it naturally, like you're explaining to a classmate.

Before you press Record

☐ Slide 1: replace `[Your name]`, `[Course and section]`, `[Instructor]`.

☐ Open the slides in the browser and go full screen on slide 1.

☐ Open **VS Code** with `tsp_brute_force.py`. Zoom in (Ctrl + `+`) until the code is easy to read, about 18–20 pt.

☐ Open a **terminal** (PowerShell or Windows Terminal) in `D:\PIT_graphtheo`, **maximized**, with a large font. Run the program once as a test, then clear the screen: `cls`.

☐ Turn on Windows "Do not disturb" so no notifications pop up. Test your microphone.

☐ Start the screen recording (Snipping Tool → Record, or OBS).

PART 1: Introduction, problem and objective (about 2:15)

▶ Slide 1: cover

Good day! I am [name] from [section]. My assigned algorithm is the Brute Force algorithm. In this video, I will use it to solve a real Traveling Salesperson Problem here in Cagayan de Oro City: the shortest one-day tourist tour that starts and ends at a hotel downtown.

▶ Slide 2: the problem

Here is the situation. A tourist is staying at Seda Centrio, a hotel in downtown Cagayan de Oro. In one day, they want to visit seven tourist spots by car, then go back to the hotel.

The question is: in what order should they visit the places, so that the total driving distance is as short as possible?

This is exactly the Traveling Salesperson Problem: start at one place, visit every required place exactly once, and return to the start, with the smallest total cost. In graph theory terms, we are looking for a Hamiltonian cycle with minimum total weight.

The quantity I will minimize is total driving distance, in kilometers. I chose distance instead of time because distance does not change much with traffic.

▶ Slide 3: the locations

These are the eight places. They will be the eight vertices of my graph.

Vertex A is the start and the end: Seda Centrio. The seven stops are: B, St. Augustine Metropolitan Cathedral; C, Museo de Oro at Xavier University; D, Plaza Divisoria; E, Limketkai Center; F, Mapawa Nature Park; G, Macahambus Hill Cave and Gorge; and H, Amaya View.

I found all of them on Google Maps. Notice that A to E are all downtown and close together, while F, G and H are far, ten to fifteen kilometers away. That is why the order really matters.

▶ Slide 4: assumptions

The brief asks us to separate actual data from assumed data.

The actual data come from Google Maps: the eight places, their coordinates, and the driving routes between every pair of places. I collected them on October 19, 2026, in the evening.

The assumptions are: the tour starts and ends at the hotel, the tourist travels by car, and for each trip I use the shortest route that Google Maps suggests. I also assume the distances do not change during the day, and I don't include opening hours or time spent at each place.

The constraints are: the tour must be closed, every stop must be visited exactly once, and the total distance must be as small as possible.

PART 2: The Brute Force algorithm (about 2:15)

▶ Slide 5: how brute force works

My assigned algorithm is Brute Force, also called exhaustive search. The idea is very simple: try every possible tour, compute the length of each one, and keep the shortest.

Step by step: One, fix the starting point, A. Two, list every possible order of the other n minus 1 places. There are n minus 1 factorial orders. Three, for each order, go back to the start to close the cycle, and add up all the distances. Four, compare it with the best tour so far, and keep the shorter one. Five, after checking every order, the best one is the optimal tour.

The formula at the bottom shows how a tour's length is computed: just the sum of the distances of its legs.

▶ Slide 6: complexity

How much work is this? There are n minus 1 factorial tours, and each tour needs n additions, so the total is n factorial. The time complexity is O of n factorial.

For my problem, n equals 8, so the program checks 7 factorial, which is 5,040 tours. That takes only about three hundredths of a second.

But look at how fast it grows. With 12 places, it is almost 40 million tours, about 7 minutes. With 15 places, about two weeks. With 20 places, tens of thousands of years. So brute force is perfect for small problems like mine, but it does not scale.

▶ **Slide 7: advantages, limitations, optimality**

Advantages: brute force always finds the exact best answer. It is easy to understand, easy to code and easy to check. It also works when the distances are different in each direction.

Limitations: it is very slow for big problems, and it wastes time checking tours that are obviously bad.

Does it guarantee the optimal solution? Yes. It is not an approximation. Since it compares every single tour, no tour can be shorter than the one it finds. This is called proof by exhaustion.

PART 3: Graph model, data and methodology (about 2:15)

▶ **Slide 8: the graph**

Now the graph model. The vertices are the eight places, A to H. An arc from u to v means driving from u to v on the road. Every place can reach every other place, so the graph is complete, with 8 times 7, or 56 arcs. The weight of each arc is the Google Maps driving distance in kilometers.

Is it directed or undirected? It is **directed**. Because of one-way streets, the distance going is often different from the distance coming back. This happened in 19 of the 28 pairs. For example, from the hotel A to Museo de Oro C is 1.40 kilometers, but from C back to A is only 1.10.

In the figure, each line has two numbers: the distance in one direction, then the distance in the other direction.

▶ **Slide 9: weight matrix**

This is the complete weight matrix, and it is exactly the input file of my program. Read it as: row is "from", column is "to".

For each pair, Google Maps usually suggested two or three routes, and I took the shortest one. For example, from A to B, Google suggested 1.8, 1.8 and 1.7 kilometers, so the weight is 1.70. All the routes are saved in my dataset so anyone can check them.

▶ **Slide 10: applying brute force**

Here is how I applied brute force to my graph. The start is fixed at A, so the program tries every order of the seven stops, B to H: 7 factorial, or 5,040 tours. Since the graph is directed, a tour and its reverse have different lengths, so all 5,040 must be checked.

Here is one calculation. The first tour is A, B, C, D, E, F, G, H, back to A. Its length is 1.70 plus 0.80 plus 0.45 plus 2.10 plus 11.50 plus 25.90 plus 18.80 plus 10.10, which is 71.35 kilometers.

The table on the right shows every time the search found a shorter tour: 71.35, then 69.05, 67.75, 64.55, 64.50, and finally 63.50 kilometers at tour number 858. After that, the remaining tours were all checked, and none was shorter.

PART 4: Python implementation and live run (about 3:45)

▶ Slide 11: code overview, say this while on the slide

My Python program is called `tsp_brute_force.py`. Let me show you the actual code.

▶ Switch to VS Code. Scroll to line 39, "# 1. INPUT DATA"

The program has five parts, marked with numbered comments.

Part 1 is the input. The function `load_locations` reads the eight places from `locations.csv`, and `load_weight_matrix` reads the distance matrix from `distance_matrix_km.csv`. ▶ point at line 55 On line 55, every distance is stored as a `Decimal`. This makes the additions exact: 0.45 plus 0.45 gives exactly 0.90, with no rounding error from the computer.

▶ Scroll to line 62, `tour_length`

Part 2 is the algorithm. The function `tour_length` takes a tour, like A, B, C, all the way back to A, and adds the distance of each leg, from `tour[i]` to `tour[i + 1]`. For our tours, that is eight distances.

▶ Scroll to line 67, `brute_force_tsp`. Highlight lines 76 to 91 with the mouse

This function, `brute_force_tsp`, is the brute force algorithm itself.

▶ point at line 76 Line 76 makes the list `others`: all the places except the start, so B to H.

▶ point at line 79 Line 79 is the most important line. `itertools.permutations(others)` generates every possible order of those seven places: all 5,040 of them. The for loop goes through each one.

▶ point at line 80 Line 80 adds the start, A, at the beginning and at the end, so the tour becomes a closed cycle.

▶ point at line 81 Line 81 computes the length of that tour.

▶ point at lines 84 and 85 Lines 84 and 85: if this is the first tour, or if it is shorter than the best so far, it becomes the new best.

▶ point at lines 88 and 89 Lines 88 and 89: if it is exactly equal to the best, we also keep it, so we know if there are ties.

▶ point at line 91 After the loop, line 91 returns the best length and the best tour. Because the loop goes through every permutation, nothing is skipped. That is why the answer is guaranteed to be optimal.

▶ Scroll to line 97, `verify_tour`

Part 3 checks the answer. `verify_tour` tests five things: the tour starts at A, ends at A, has seven stops, no stop is repeated, and every stop is included.

▶ Scroll to line 141, `main`

Part 4 only prints tables, and Part 5, `main`, runs everything in order: read the data, show it, run the search, then show the result, the checks, and a comparison. Now let me run it.

▶ Switch to the terminal. Type this and press Enter:

```
type data\distance_matrix_km.csv
```

First, this is the input data file: the distance matrix in kilometers from Google Maps, the same numbers as on my slide.

▶ Type this and press Enter:

```
python tsp_brute_force.py
```

It finished instantly. Let me scroll up to the top and go through the output.

▶ Scroll to the top: "INPUT 1" and "INPUT 2"

First, the program shows the input: the eight vertices with their Google Maps coordinates, and the weight matrix. Row is "from", column is "to".

▶ Scroll down: "ALGORITHM" and "First 5 tours"

It says the number of tours to check is 7 factorial, 5,040. Then it shows the first five tours, added edge by edge. For example, tour number 1 is 71.35 kilometers, the same as my hand calculation.

▶ Scroll down: "Running the exhaustive search"

During the search, it prints every time it finds a shorter tour: 71.35, 69.05, 67.75, 64.55, 64.50, and finally 63.50 at tour number 858.

▶ Scroll down: "RESULT"

Here is the result. Tours evaluated: 5,040, which matches 7 factorial, so all tours were checked. The search took ▶ read the time on your screen seconds. The shortest length is 63.50 kilometers, and there is only one optimal tour: A, C, D, B, G, H, F, E, back to A.

▶ Scroll down: "VERIFICATION 1"

Verification 1 checks that it is a valid tour. All five checks pass, and the visit count shows each stop B to H was visited exactly once.

▶ Scroll down: "VERIFICATION 2"

Verification 2 recomputes the total from the edge weights, leg by leg. The running total goes 1.40, 1.85, 2.30, 15.40, 34.20, 51.00, 62.30, and ends at 63.50, which matches the search result.

▶ Scroll down: "COMPARISON"

Finally, the comparison. If we visited the places in the order I listed them, the tour would be 71.35 kilometers. The average of all tours is 72.99, and the worst is 78.10. So the optimal route saves about 9.5 kilometers compared with an average tour. The next best tour is 64.05 kilometers.

▶ Switch back to the slides, slide 13

PART 5: Results and conclusion (about 2:45)

▶ Slide 13: the optimal tour

So the shortest tour is 63.50 kilometers. Leaving the hotel, the tourist goes to Museo de Oro, then Plaza Divisoria, then St. Augustine Cathedral. These three are very close: less than half a kilometer apart. Then they drive south to Macahambus, then to Amaya View, then east to Mapawa Nature Park, then to Limketkai Center, and finally back to the hotel.

▶ Slide 14: checking the answer

To make sure the answer is correct, I checked it in several ways. The tour starts and ends at A, every stop is visited exactly once, all 5,040 tours were checked, and the total recomputed from the matrix is 63.50. I also solved the same problem with a different exact method, Held–Karp dynamic programming, and it found the same tour. Google rounds its distances, so I also checked that the rounding cannot change the winner.

▶ Slide 15: savings

This chart shows the lengths of all 5,040 tours. Only one tour reaches 63.50 kilometers. Compared with an average tour, the optimal order saves 9.49 kilometers, or 13 percent.

▶ Slide 16: conclusion

To conclude: the shortest tourist day tour from Seda Centrio is A, C, D, B, G, H, F, E, and back to A, with 63.50 kilometers. On the real map it makes sense: visit the downtown places first, then do one big loop through the far places, and stop at Limketkai on the way home.

This route is proven optimal, not just an approximation, because brute force compared all 5,040 possible tours and none is shorter. This is true for the Google Maps distances I collected on October 19, 2026.

▶ Slide 17: limitations

My work has some limitations. First, the distances are from one moment in time, and Google's routes can change with traffic. Second, the way I define the distance matters: when I used Google's first suggested route instead of the shortest one, the best tour became the same loop but in the opposite direction. Third, I only minimized distance, not time, opening hours or cost. And brute force only works because I have 8 places; with 15 places it would take about two weeks.

To improve this, we could minimize travel time or cost instead, add opening hours, collect the distances at the actual time of the trip, and for more places use a faster method like Held–Karp or a heuristic.

▶ Slide 18: references

These are my references, including the paper by Luna-Garcia, Candano and Caga-anan, and Google Maps for the data. My Python program and dataset are submitted together with this video. Thank you!

▶ Stop recording

If you're running over 15 minutes, shorten these first

1. Slide 6: skip the 12/15/20 rows and just say "it grows factorially".
2. Slide 15: one sentence ("only one tour reaches 63.50; it saves 13 percent").
3. Live run: skip `type data\distance_matrix_km.csv` and just point at INPUT 2 in the output.

If the teacher asks questions

- **"Why not divide 5,040 by 2?"** In an undirected graph a tour and its reverse have the same length, so you could halve the count. Our graph is directed, so the reverse is a different tour with a different length (for example, the reverse of our best tour is 65.85 km).
- **"Is it really optimal?"** Yes, for this data. Brute force checked every tour, and Held–Karp, a second exact method, gives the same answer.
- **"Where do the numbers come from?"** Google Maps driving directions for every ordered pair, collected 19 Oct 2026. Each weight is the shortest suggested route. Everything is logged in `data/raw/` and `data/routes_all.csv`.